

Submission deadline and procedure: refer to Blackboard.

Introduction

This is the second (of two) assessed exercise for 6CM504. This exercise is worth a total of 50% of the module.

Submission – all students

Your submission for this exercise should consist of a source CSPM script (exercise2_username.csp) and any accompanying documentation (such as the short report discussed in the lecture). Submission should be via the “Assessment 2” portal on Blackboard. The deadline for submission is given on Blackboard. You will have approximately two weeks to complete this assignment (including two lab sessions) – it is not expected that the lab sessions alone will allow enough time to develop your solution.

This is an individual exercise. The University of Derby operates strict rules regarding plagiarism and these will be enforced where necessary. You may not use generative AI in constructing your solutions – indeed it will not be helpful to do so.

Background

In the lectures, we introduced several industrial case studies – modelling protocols, modelling component failure, and modelling source code. We demonstrated how CSP could be used to build models of various systems, and FDR used to analyse properties of interest. In this lab exercise you will develop these models further and explore how we improve our analysis – both to confirm properties of robustness, and to expose problematic scenarios.

The Task

In lectures we presented Dekker’s algorithm and some code that implemented Dekker’s algorithm. This algorithm implements a mutual exclusion protocol for two processes, using only shared memory.

In the first part of this exercise, you should ensure you have a working model of Dekker’s algorithm in FDR, and that you can verify that your model is correct using an appropriate safety property modelled as traces refinement assertion. In this context, “correct” means that both processes cannot be in the critical section at the same time.

Modern compilers are very clever pieces of software – they can optimise code and introduce many efficiencies that are very difficult to implement, or to spot, at source code level. For instance, it can be possible to detect that a variable is not accessed within a particular scope and for that variable to be automatically removed to give a space efficiency. Normally, this produces an entirely equivalent piece of code with the exception of the improved space efficiency – but this is not always the case in a shared memory environment.

This is a particular problem for Dekker’s algorithm. In some environments it is – surprisingly – perfectly legal for a compiler to detect that the variables `flag1` and `flag2` are never accessed in the loop and therefore remove the writes to those variables in each loop. Similarly, it is also legal

Submission deadline and procedure: refer to Blackboard.

for a compiler to detect that the variable `turn` is not accessed in the loop and perform a similar transformation. Clearly, the issue here is that the compiler has made some assumptions (erroneously as it turns out) and performed a transformation on the code that firstly means it can break, and secondly means the compiled version is not faithful to the original source code.

In a case such as this it is reasonable to investigate the optimisations a compiler may make and understand if they have any effect – particularly an erroneous effect – on the final system. For this next task you are required to adapt your original model of Dekker’s algorithm to reflect the possible compiler optimisations discussed above, and prove that the algorithm will break when these optimisations are turned on. Your model should consider the possibility that one, or both, or these optimisations may, or may not, have been turned on.